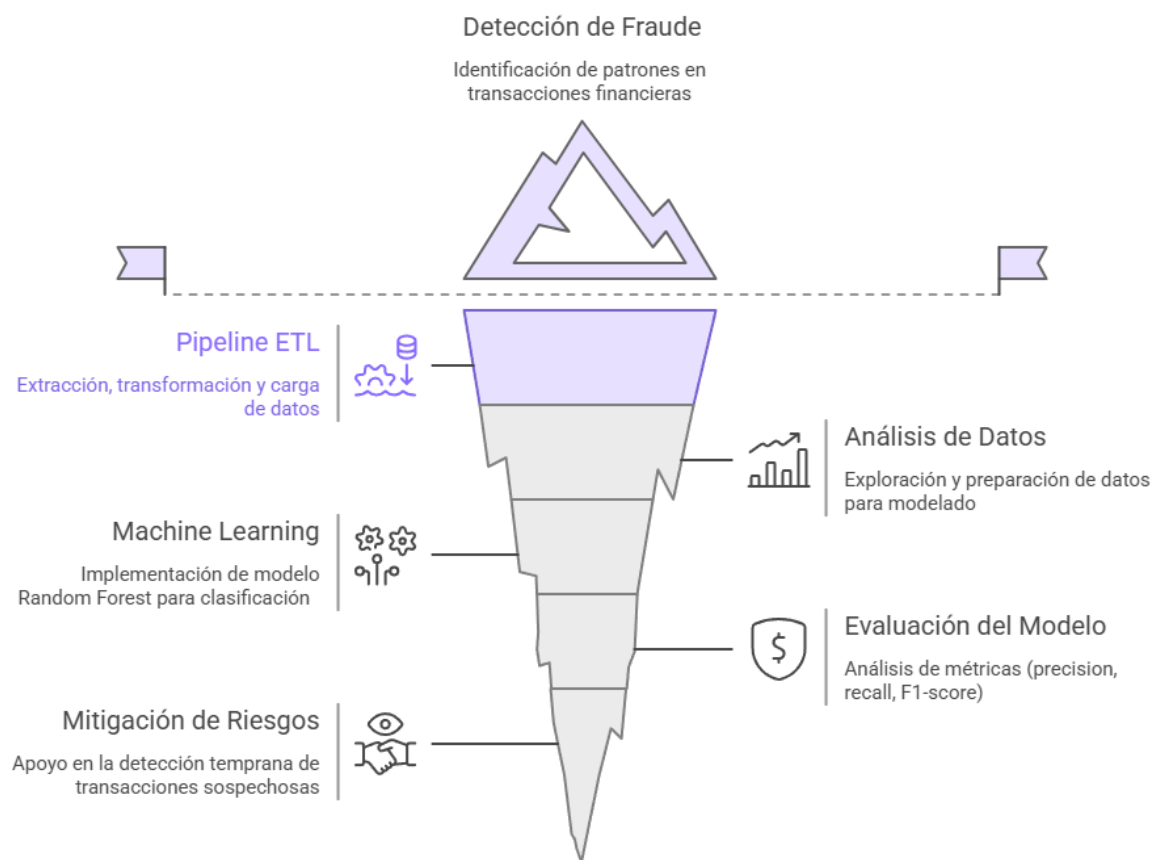


PROYECTO DE PIPELINE ETL Y MODELO PREDICTIVO PARA DETECCIÓN DE FRAUDE

Python · Pandas · Scikit-learn · Random Forest · Logging

Detección de Fraude: Más allá de la Superficie



Desarrollado por:

Tolentino Tarazona Dany

20 de abril de 2026

Índice

1.	Introducción al problema	4
2.	Arquitectura de la solución.....	5
	Componentes principales.....	5
	Enfoque de diseño.....	5
	Flujo dentro de la arquitectura	5
3.	Pipeline de datos	6
	Flujo de procesamiento.....	7
	Procesos clave del ETL.....	7
	Enfoque de calidad y trazabilidad	10
4.	Modelo de Machine Learning.....	11
	Modelo seleccionado	11
	Justificación de la elección	12
	Variables de entrada.....	12
	Manejo del desbalance de datos	13
	Proceso de entrenamiento	13
	Pipeline de predicción	14
5.	Resultados	14
	Métricas de evaluación	14
	Interpretación de resultados.....	15
	Valor para el negocio.....	16
	Limitaciones del modelo	16
6.	Buenas prácticas implementadas	16
	Modularización del código	16
	Separación de capas de datos	17
	Uso de logging.....	17
	Reproducibilidad del modelo	18
	Organización del pipeline	18
7.	Mejoras futuras	19
	Escalabilidad del procesamiento de datos	19
	Migración a entornos Cloud	19
	Automatización del pipeline.....	19
	Despliegue del modelo (Model Serving)	19
	Mejora del modelo	20
8.	Conclusión	20

1. Introducción al problema

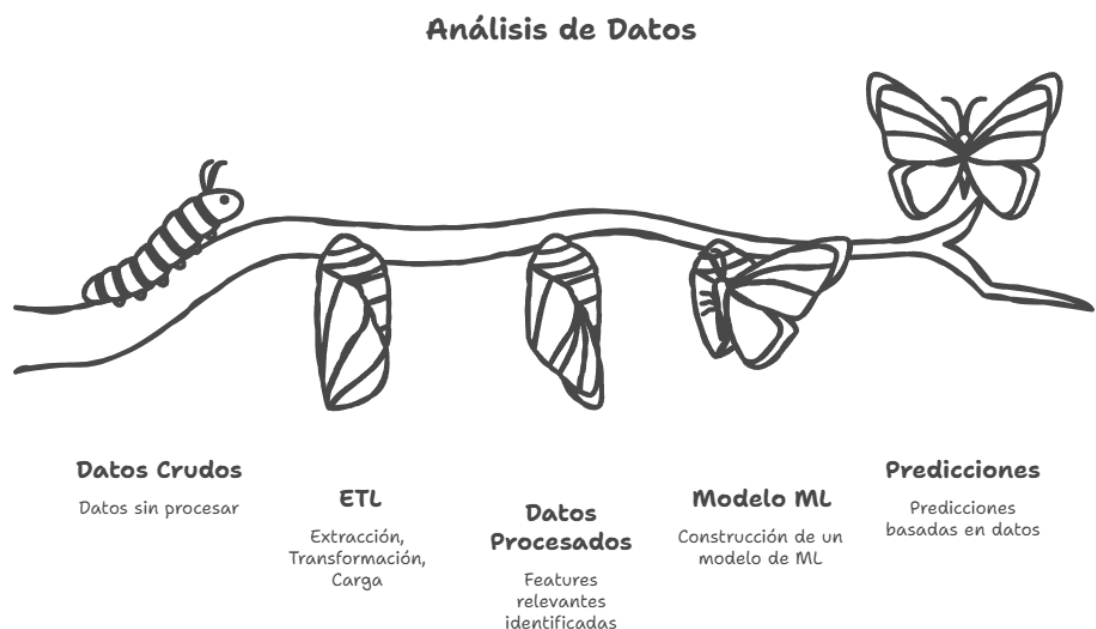
El crecimiento de las transacciones digitales ha incrementado significativamente la exposición al fraude financiero, convirtiéndolo en un desafío crítico para instituciones bancarias y plataformas de pago. La detección de este tipo de actividades resulta compleja debido al alto volumen de datos, la variabilidad del comportamiento de los usuarios y el desbalance entre transacciones legítimas y fraudulentas.

En este contexto, el presente proyecto desarrolla una solución basada en datos que integra un pipeline ETL con un modelo de Machine Learning, con el objetivo de identificar patrones asociados a posibles fraudes en transacciones financieras.

La solución implementada permite transformar datos crudos en información procesada mediante un flujo estructurado de extracción, transformación y carga. Posteriormente, se aplican técnicas de feature engineering para generar variables relevantes que alimentan un modelo de clasificación basado en Random Forest, el cual es capaz de detectar comportamientos anómalos.

Como resultado, el sistema permite automatizar el análisis de datos transaccionales y mejorar la capacidad de detección de fraude, aportando una base sólida para la toma de decisiones en entornos financieros.

Este proyecto demuestra la aplicación práctica de principios de ingeniería de datos y machine learning en un flujo completo, desde la ingesta de datos hasta la generación de predicciones, manteniendo una estructura modular y escalable alineada a escenarios reales.



2. Arquitectura de la solución

La solución ha sido diseñada bajo un enfoque modular, en el cual cada componente del sistema cumple una función específica dentro del flujo de procesamiento de datos. Esta organización permite separar responsabilidades, mejorar la mantenibilidad del código y facilitar la escalabilidad del proyecto.

A nivel general, la arquitectura se divide en capas que representan las distintas etapas del ciclo de vida de los datos: desde su almacenamiento inicial hasta su consumo por el modelo de machine learning. Esta estructura no solo ordena el desarrollo, sino que también refleja prácticas utilizadas en entornos reales de ingeniería de datos.

Componentes principales

El proyecto se organiza en los siguientes módulos:

- Data: gestiona los datos en diferentes etapas (crudos, procesados y predicciones), permitiendo trazabilidad y control del flujo de información.
- ETL: contiene el pipeline de procesamiento de datos, donde se realizan las tareas de limpieza, transformación, validación y carga.
- Features: encapsula la lógica de generación de variables derivadas, separando esta responsabilidad del ETL para mantener consistencia en el modelado.
- Models: incluye los procesos de entrenamiento y predicción, así como la persistencia del modelo y sus artefactos.
- Pipelines: actúa como punto de entrada del sistema, orquestando la ejecución de todas las etapas de manera secuencial.

Enfoque de diseño

Uno de los principios clave en esta arquitectura es la separación de responsabilidades, donde cada módulo opera de manera independiente pero coordinada. Esto permite:

- Modificar componentes sin afectar todo el sistema
- Reutilizar partes del código en otros proyectos
- Escalar la solución de manera progresiva

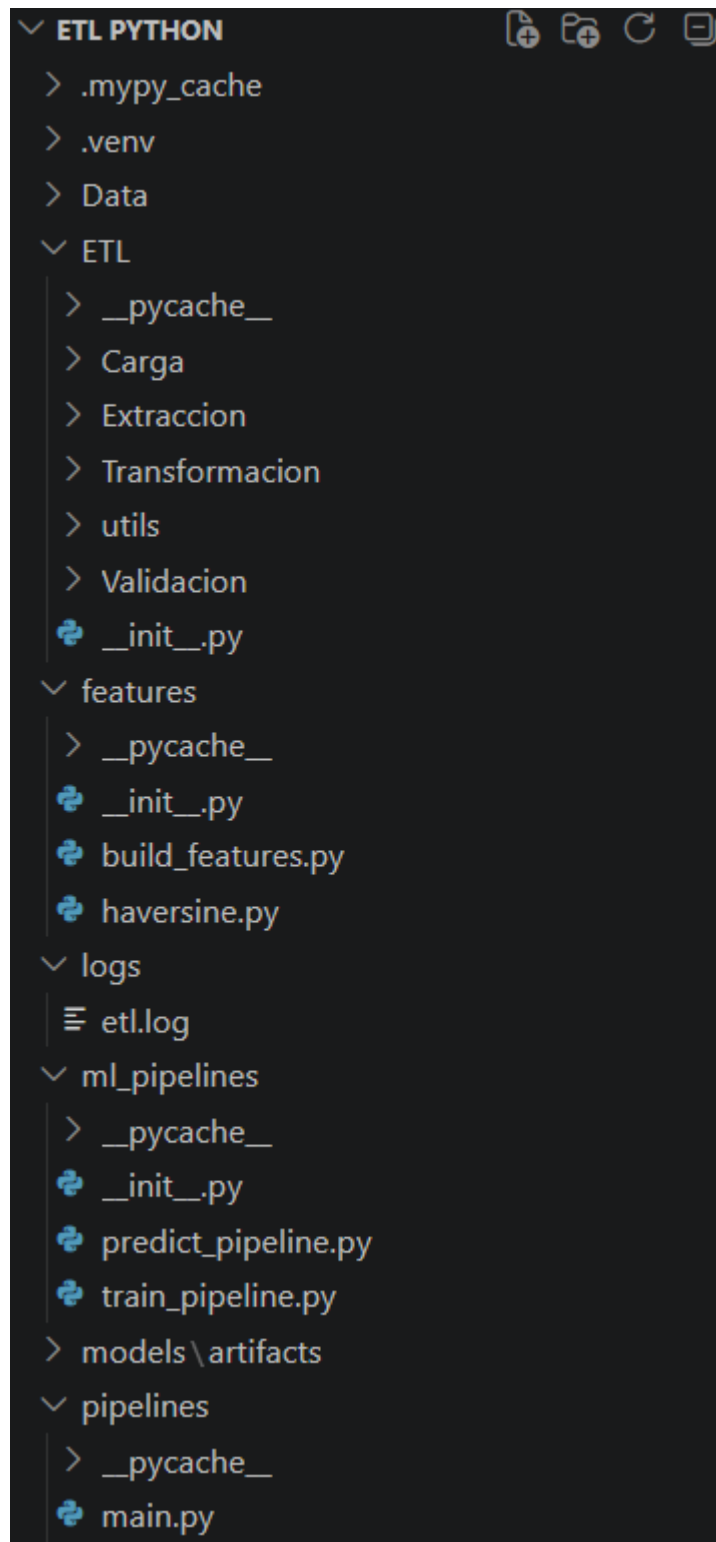
Adicionalmente, el uso de una estructura organizada por capas facilita la comprensión del flujo de datos y reduce la complejidad del sistema.

Flujo dentro de la arquitectura

El sistema sigue un flujo continuo donde cada módulo transforma la información antes de pasarla al siguiente:

- Los datos se almacenan inicialmente en la capa Data (raw)
- Son procesados por el módulo ETL
- Se enriquecen en la capa de Features
- Se utilizan en Models para entrenamiento y predicción

Este flujo garantiza que el modelo opere sobre datos limpios, estructurados y enriquecidos, lo cual es fundamental para obtener resultados confiables.



3. Pipeline de datos

El pipeline ETL constituye el núcleo del sistema, siendo responsable de transformar datos transaccionales en bruto en información estructurada y lista para su uso en el modelo de machine learning.

En lugar de implementar un flujo monolítico, se diseñó un proceso modular que permite aplicar transformaciones de manera controlada y reutilizable. Este enfoque

facilita la mantenibilidad del código y asegura que cada etapa del procesamiento pueda evolucionar de forma independiente.

Flujo de procesamiento

El pipeline sigue una secuencia lógica orientada a garantizar la calidad de los datos:

1. Datos crudos
2. Limpieza
3. Transformación
4. Validación
5. Datos procesados

A lo largo de este flujo, los datos pasan por distintas etapas donde se eliminan inconsistencias, se estandarizan formatos y se preparan para su posterior análisis.

Procesos clave del ETL

1. Limpieza de datos

Se eliminan columnas irrelevantes o redundantes que no aportan valor al análisis, reduciendo el ruido en el dataset y optimizando el procesamiento.

Ejemplo de código:

```
import pandas as pd
from ETL.utils.logger import get_logger
import ETL.utils.config as config
lg = get_logger()
def delete(df:list[dict[str,pd.DataFrame]]) -> list[dict[str,pd.DataFrame]]:
    try:
        lg.info("Iniciando a eliminar columnas")
        df_deleted = []
        dct_deleted = {}
        delete_column = []
        for data in df:
            for file_name, datF in data.items():
                for col in datF.columns:
                    if col not in config.COLUMNS_TO_KEEP:
                        delete_column.append(col)
                lg.info(f"{file_name}:Columns eliminadas {list(delete_column)}")
                cleaned_df = datF.drop(columns=delete_column)
                dct_deleted[file_name] = cleaned_df
                delete_column.clear()
        df_deleted.append(dct_deleted)
        lg.info("Eliminación exitosa de columnas")
        return df_deleted
    except Exception as e:
        raise ValueError(f"Error al transformar datos: {e}")
```

Esta etapa permite trabajar con un conjunto de datos más enfocado y eficiente.

2. Estandarización

Se aplican transformaciones para asegurar consistencia en los datos

```
import pandas as pd
from ETL.utils.logger import get_logger
lg = get_logger()
def standardize(df: list[dict[str, pd.DataFrame]]) -> list[dict[str, pd.DataFrame]]:
    try:
        lg.info("Iniciando estandarización")
        df_standardized = []
        dct_standardized = {}
        for data in df:
            for file_name, datF in data.items():
                #Estandarizar nombres de columnas
                datF.columns = datF.columns.str.strip().str.lower().str.replace(" ", "_")
                #fechas
                colum_fechas = datF.filter(like="date").columns
                for col in colum_fechas:
                    datF[col] = pd.to_datetime(datF[col], errors="coerce")
                dct_standardized[file_name]=datF
                lg.info(f"{file_name}: Estandarización correcta de las columnas {datF.columns}")

        df_standardized.append(dct_standardized)
        lg.info("Estandarización exitosa")
        return df_standardized
    except Exception as e:
        raise ValueError(f"Error al estandarizar datos: {e}")
```

Esto garantiza que las etapas posteriores operen sobre datos homogéneos.

3. Transformación y enriquecimiento


```

import pandas as pd
from ETL.utils.logger import get_logger
lg = get_logger()
def adding(df:list[dict[str,pd.DataFrame]]) -> list[dict[str,pd.DataFrame]]:
    try:
        lg.info("Iniciando agregación de columnas")
        df_added = []
        dict_added = {}
        for data in df:
            for file_name, datF in data.items():
                #Agregar columna de año
                datF["year"] = datF["trans_date_trans_time"].dt.year
                #Agregar columna de mes
                datF["month"] = datF["trans_date_trans_time"].dt.month
                #Agregar columna de día de semana
                datF["day"] = datF["trans_date_trans_time"].dt.dayofweek
                #Agregar columna de día del mes
                datF["day_month"] = datF["trans_date_trans_time"].dt.day
                #Agregar columna de hora
                datF["hour"] = datF["trans_date_trans_time"].dt.hour
                #Agregar booleano de transacciones nocturnas
                datF["is_night"] = datF["hour"].apply(lambda x: 1 if x >= 22 or x < 6 else 0)
                # Convertimos a datetime y extraemos solo la fecha
                datF["trans_date"] = pd.to_datetime(datF["trans_date_trans_time"]).dt.date
                # Calcular el promedio de monto por cliente
                datF["avg_amt_client"] = datF.groupby("cc_num")["amt"].transform("mean")
                # Calcular el promedio de transacciones por cliente al día
                datF["transactions_per_day"] = datF.groupby(["cc_num", "trans_date"])["amt"].transform("count")
                dict_added[file_name] = datF
            lg.info(f"{file_name}: Agregación de columnas correcta")
        df_added.append(dict_added)
        lg.info("Agregación de columnas exitosa")
        return df_added
    except Exception as e:
        raise ValueError(f"Error al agregar columnas: {e}")

```

Se generan nuevas columnas o se modifican existentes para mejorar la calidad del dataset. Estas transformaciones preparan la información para el proceso de feature engineering.

4. Validación de columnas

```

import pandas as pd
from ETL.utils.logger import get_logger
import ETL.utils.config as config
lg = get_logger()
def verify_columns(df:list[dict[str,pd.DataFrame]]) -> list[dict[str,pd.DataFrame]]:
    try:
        lg.info("Iniciando verificación de columnas")
        for data in df:
            missing_columns = []
            for file_name, dataF in data.items():
                for col in config.COLUMNS_TO_KEEP:
                    if col not in dataF.columns:
                        missing_columns.append(col)
            if missing_columns:
                raise ValueError(f"Faltan columnas: {list(missing_columns)} en el DataFrame {file_name}")
            lg.info(f"Verificación de columnas exitosa en {file_name}")
        lg.info("Verificación de columnas exitosa")
        return df
    except Exception as e:
        raise ValueError(f"Error en la verificación de columnas: {e}")

```

5. Carga de datos

Finalmente, los datos transformados se almacenan en la capa processed, permitiendo su reutilización en etapas posteriores del sistema.

```
import pandas as pd
from ETL.utils.logger import get_logger
from ETL.utils.config import destino
lg = get_logger()
def load(df : list[dict[str,pd.DataFrame]]) -> None:
    try:
        lg.info("Iniciando carga de datos")
        for data in df:
            for file_name, datF in data.items():
                destino_path = f"{destino}/{file_name}"
                datF.to_csv(destino_path, index=False)
                lg.info(f"{file_name}: Carga exitosa en formato CSV")
            lg.info("Carga de datos exitosa")
    except Exception as e:
        raise ValueError(f"Error al cargar datos: {e}")
```

Enfoque de calidad y trazabilidad

El pipeline ETL no solo transforma datos, sino que también establece una base confiable para el resto del sistema. La incorporación de validaciones y la separación por etapas permite:

- Detectar errores de forma temprana
- Mantener trazabilidad sobre el flujo de datos
- Garantizar consistencia entre entrenamiento y predicción.

```

2026-04-16 10:59:56,748 - INFO - -----
2026-04-16 10:59:56,749 - INFO - Iniciando PIPELINE
2026-04-16 10:59:56,749 - INFO - Descubriendo archivos en C:\Users\dnayj\Desktop\DATA practica\ETL PYTHON\Data\raw
2026-04-16 10:59:56,749 - INFO - Archivos descubiertos: ['fraudTest.csv', 'fraudTrain.csv']
2026-04-16 10:59:56,749 - INFO - Iniciando extracci3n
2026-04-16 10:59:59,902 - INFO - Extracci3n exitosa de fraudTest.csv
2026-04-16 11:00:06,205 - INFO - Extracci3n exitosa de fraudTrain.csv
2026-04-16 11:00:06,205 - INFO - Iniciando verificaci3n de columnas
2026-04-16 11:00:06,205 - INFO - Verificaci3n de columnas exitosa en fraudTest.csv
2026-04-16 11:00:06,205 - INFO - Verificaci3n de columnas exitosa en fraudTrain.csv
2026-04-16 11:00:06,205 - INFO - Verificaci3n de columnas exitosa
2026-04-16 11:00:06,205 - INFO - Iniciando transformaci3n
2026-04-16 11:00:06,205 - INFO - Iniciando a eliminar columnas
2026-04-16 11:00:06,205 - INFO - fraudTest.csv:Columns eliminadas ['first', 'last', 'gender', 'street', 'city', 'state', 'zip', 'job']
2026-04-16 11:00:06,214 - INFO - fraudTrain.csv:Columns eliminadas ['first', 'last', 'gender', 'street', 'city', 'state', 'zip', 'job']
2026-04-16 11:00:06,222 - INFO - Eliminaci3n exitosa de columnas
2026-04-16 11:00:06,223 - INFO - Iniciando estandarizaci3n
2026-04-16 11:00:06,466 - INFO - fraudTest.csv: Estandarizaci3n correcta de las columnas Index(['trans_date_trans_time', 'cc_num', 'merchant', 'category', 'amt', 'lat',
'long', 'city_pop', 'dob', 'trans_num', 'unix_time', 'merch_lat',
'merch_long', 'is_fraud'],
dtype='str')
2026-04-16 11:00:06,950 - INFO - fraudTrain.csv: Estandarizaci3n correcta de las columnas Index(['trans_date_trans_time', 'cc_num', 'merchant', 'category', 'amt', 'lat',
'long', 'city_pop', 'dob', 'trans_num', 'unix_time', 'merch_lat',
'merch_long', 'is_fraud'],
dtype='str')
2026-04-16 11:00:06,950 - INFO - Estandarizaci3n exitosa
2026-04-16 11:00:06,950 - INFO - Iniciando agregaci3n de columnas
2026-04-16 11:00:07,510 - INFO - fraudTest.csv: Agregaci3n de columnas correcta
2026-04-16 11:00:09,113 - INFO - fraudTrain.csv: Agregaci3n de columnas correcta
2026-04-16 11:00:09,113 - INFO - Agregaci3n de columnas exitosa
2026-04-16 11:00:09,113 - INFO - Transformaci3n exitosa
2026-04-16 11:00:09,207 - INFO - Iniciando carga de datos
2026-04-16 11:00:16,876 - INFO - fraudTest.csv: Carga exitosa en formato CSV
2026-04-16 11:00:33,516 - INFO - fraudTrain.csv: Carga exitosa en formato CSV
2026-04-16 11:00:33,516 - INFO - Carga de datos exitosa
2026-04-16 11:00:33,516 - INFO - PIPELINE finalizado exitosamente

```

4. Modelo de Machine Learning

Una vez generado el conjunto de datos enriquecido mediante el proceso de feature engineering, se procede al entrenamiento de un modelo de machine learning orientado a la detección de fraude en transacciones financieras.

El objetivo de este modelo es clasificar cada transacción como fraudulenta o no fraudulenta, identificando patrones complejos que no son evidentes mediante reglas tradicionales.

Modelo seleccionado

Para este proyecto se utilizó un modelo de tipo Random Forest Classifier, un algoritmo basado en múltiples árboles de decisión que permite capturar relaciones no lineales dentro de los datos.

Ejemplo de implementación:

```
model = RandomForestClassifier(  
    n_estimators=100,  
    max_depth=10,  
    class_weight="balanced",  
    random_state=42,  
    n_jobs=-1  
)
```

Justificación de la elección

El modelo fue seleccionado por las siguientes razones:

- Buen desempeño en datos tabulares, como los utilizados en transacciones financieras
- Capacidad para manejar relaciones complejas entre variables
- Robustez frente al overfitting, gracias a su naturaleza basada en múltiples árboles
- Facilidad de implementación y entrenamiento, lo cual lo hace ideal para prototipos funcionales

Variables de entrada

El modelo utiliza como entrada:

Variables procesadas provenientes del pipeline ETL

Features generadas en la etapa de feature engineering (como la distancia geográfica)

```
import numpy as np
def haversine(lat1, lon1, lat2, lon2):
    R = 6371 # Radio de la Tierra en km

    lat1 = np.radians(lat1)
    lon1 = np.radians(lon1)
    lat2 = np.radians(lat2)
    lon2 = np.radians(lon2)

    dlat = lat2 - lat1
    dlon = lon2 - lon1

    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    c = 2 * np.arcsin(np.sqrt(a))

    return R * c
```

Esto permite que el modelo trabaje con información más rica y representativa del comportamiento de las transacciones.

Manejo del desbalance de datos

Uno de los principales retos en la detección de fraude es el desbalance en las clases, donde las transacciones fraudulentas representan un porcentaje muy bajo del total.

Para abordar este problema, se utiliza el parámetro:

```
class_weight="balanced"
```

Esto permite ajustar automáticamente el peso de cada clase durante el entrenamiento, dando mayor importancia a la detección de fraudes.

Proceso de entrenamiento

El entrenamiento del modelo se encuentra encapsulado en un pipeline dedicado, lo que permite estructurar el proceso de manera clara y reproducible.

Durante esta etapa se realiza:

- Selección de variables de entrada
- Entrenamiento del modelo
- Persistencia del modelo entrenado en formato .pkl
- Almacenamiento de metadatos (como columnas utilizadas)

Este enfoque permite reutilizar el modelo en etapas posteriores sin necesidad de reentrenarlo.

Pipeline de predicción

El proyecto incluye un pipeline de inferencia que permite aplicar el modelo entrenado sobre nuevos datos.

Este proceso consiste en:

- Cargar el modelo previamente entrenado
- Aplicar las mismas transformaciones de datos
- Generar predicciones para nuevas transacciones

Esto simula un flujo real donde el modelo puede ser utilizado en escenarios productivos.

5. Resultados

Una vez entrenado el modelo, se procede a evaluar su desempeño con el objetivo de medir su capacidad para identificar correctamente transacciones fraudulentas y no fraudulentas.

Dado que el problema de fraude presenta un alto desbalance de clases, la evaluación no se centra únicamente en la exactitud (accuracy), sino en métricas que permitan analizar con mayor precisión el comportamiento del modelo frente a la clase minoritaria.

Métricas de evaluación

Para medir el rendimiento del modelo se consideran principalmente:

- Accuracy: proporción de predicciones correctas sobre el total
- Recall (sensibilidad): capacidad del modelo para identificar correctamente los casos de fraude
- Precision: proporción de predicciones de fraude que son correctas

En problemas de fraude, el recall adquiere especial relevancia, ya que permite evaluar qué tan efectivo es el modelo detectando transacciones fraudulentas, incluso a costa de aceptar algunos falsos positivos.

```
model.fit(X_train, y_train)
```

```
y_probs = model.predict_proba(X_test)[: , 1]
```

```
y_pred = (y_probs > 0.7).astype(int)
```

```
print(classification_report(y_test, y_pred))
```

```
python -m ml_pipelines.train_pipeline
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	257815
1	0.52	0.84	0.65	1520
accuracy			0.99	259335
macro avg	0.76	0.92	0.82	259335
weighted avg	1.00	0.99	1.00	259335

Interpretación de resultados

El modelo presenta un desempeño general alto, alcanzando una precisión global (accuracy) cercana al 99%. Sin embargo, debido al fuerte desbalance de clases en el problema, es necesario analizar métricas más específicas para la clase minoritaria (fraude).

En este contexto, el modelo obtiene:

- Recall de 0.84 para la clase fraudulenta, lo que indica que es capaz de detectar aproximadamente el 84% de las transacciones fraudulentas.
- Precision de 0.52 para fraude, lo que significa que cerca del 52% de las transacciones clasificadas como fraude son efectivamente fraudulentas.

Este comportamiento refleja un trade-off común en problemas de detección de fraude: el modelo prioriza la identificación de la mayor cantidad posible de casos fraudulentos (alto recall), incluso a costa de generar algunos falsos positivos.

Desde una perspectiva práctica, esto implica que:

- El modelo es efectivo detectando fraude, reduciendo el riesgo de que transacciones fraudulentas pasen desapercibidas.
- Existe un nivel moderado de falsas alertas, lo cual es aceptable en sistemas de monitoreo, donde es preferible revisar transacciones adicionales antes que omitir fraudes reales.

Adicionalmente, el modelo muestra un desempeño casi perfecto en la clase no fraudulenta, lo cual es consistente con la alta proporción de transacciones legítimas en el dataset.

La incorporación de variables derivadas, como la distancia geográfica entre cliente y comercio, así como variables de comportamiento transaccional, contribuye significativamente a la capacidad del modelo para identificar patrones anómalos.

En conjunto, estos resultados demuestran que el sistema puede:

- Detectar comportamientos inusuales en las transacciones
- Identificar posibles fraudes con una alta tasa de cobertura
- Servir como un mecanismo de apoyo en sistemas de alerta temprana

En escenarios reales, este tipo de modelo prioriza la detección de fraude (recall) sobre la precisión, ya que el costo de no detectar una transacción fraudulenta es significativamente mayor.

Valor para el negocio

Desde una perspectiva aplicada, los resultados obtenidos permiten:

- Reducir el riesgo de pérdidas económicas asociadas a fraude
- Mejorar los procesos de detección automatizada
- Priorizar la revisión de transacciones sospechosas

Aunque el modelo no reemplaza completamente la validación humana, sí actúa como una herramienta de apoyo que incrementa la eficiencia en la detección.

Limitaciones del modelo

Es importante considerar ciertas limitaciones en la solución implementada:

- El desempeño depende directamente de la calidad y representatividad de los datos
- El desbalance de clases puede generar falsos positivos o negativos
- No se ha realizado un ajuste avanzado de hiperparámetros
- El modelo no está desplegado en un entorno productivo

Reconocer estas limitaciones permite tener una visión realista del alcance del proyecto y abre oportunidades de mejora.

6. Buenas prácticas implementadas

El desarrollo de este proyecto no se limita únicamente a la construcción de un pipeline funcional, sino que incorpora buenas prácticas orientadas a la mantenibilidad, escalabilidad y organización del código. Estas prácticas son fundamentales en entornos reales, donde los sistemas deben ser robustos, comprensibles y fáciles de extender.

Modularización del código

El proyecto ha sido estructurado en módulos independientes según su responsabilidad (ETL, features, modelos, pipelines), evitando la implementación de scripts monolíticos.

Este enfoque permite:

- Desarrollar y probar cada componente de forma aislada
- Reducir el acoplamiento entre partes del sistema
- Facilitar la reutilización del código en otros proyectos

La separación entre ETL, feature engineering y modelado refleja una arquitectura alineada con prácticas profesionales en proyectos de datos.

Separación de capas de datos

Se implementa una organización de datos por niveles:

- raw: datos originales sin modificar
- processed: datos limpios y transformados
- predictions: resultados generados por el modelo

Esta estructura permite mantener la integridad de los datos originales, garantizar trazabilidad y facilitar auditorías o reprocesamientos en caso de ser necesario.

Uso de logging

El proyecto incorpora un sistema de logging que permite registrar eventos durante la ejecución del pipeline.

Esto aporta:

- Visibilidad sobre el flujo de ejecución
- Facilidad para detectar errores
- Mejora en el proceso de debugging

El uso de logs es una práctica clave en sistemas productivos, donde no siempre es posible monitorear manualmente cada ejecución.

```

import os
import logging as lg
def get_logger():
    ubicacion = os.path.dirname(__file__)
    ub_etl = os.path.dirname(ubicacion)
    ub_raiz = os.path.dirname(ub_etl)
    ud_destino = os.path.join(ub_raiz, "logs")
    if not os.path.exists(ud_destino):
        os.mkdir(ud_destino)
    logger = lg.getLogger("etl")
    logger.setLevel(lg.INFO)
    log_destino = os.path.join(ud_destino, "etl.log")
    if not logger.handlers:
        handler = lg.FileHandler(log_destino)
        format = lg.Formatter("%(asctime)s - %(levelname)s - %(message)s")
        handler.setFormatter(format)
        logger.addHandler(handler)
    return logger

```

Reproducibilidad del modelo

Para garantizar consistencia en los resultados, se han aplicado prácticas como:

- Uso de random_state en el modelo
- Persistencia del modelo entrenado en archivos .pkl
- Almacenamiento de las columnas utilizadas en el entrenamiento

Esto permite que el modelo pueda ser reutilizado en distintos entornos sin variaciones inesperadas en su comportamiento.

Organización del pipeline

El uso de un archivo principal de ejecución (main.py) permite centralizar la orquestación del flujo completo, integrando:

- Procesamiento de datos (ETL)
- Generación de features
- Entrenamiento del modelo
- Predicción

Esto simula un entorno real donde los procesos son ejecutados de forma automatizada y controlada.

7. Mejoras futuras

Si bien la solución desarrollada cumple con el objetivo de construir un pipeline funcional de datos y un modelo predictivo para la detección de fraude, existen diversas oportunidades de mejora orientadas a su evolución hacia un entorno productivo.

Estas mejoras están enfocadas en escalabilidad, automatización y despliegue, aspectos clave en sistemas de datos reales.

Escalabilidad del procesamiento de datos

Actualmente, el pipeline utiliza Pandas, lo cual es adecuado para volúmenes de datos moderados. Sin embargo, en escenarios reales donde se manejan grandes volúmenes de transacciones, sería recomendable migrar a tecnologías distribuidas como PySpark.

Esto permitiría:

- Procesar grandes volúmenes de datos de forma eficiente
- Distribuir la carga de trabajo
- Reducir tiempos de procesamiento

Migración a entornos Cloud

Una evolución natural del proyecto sería su implementación en la nube, utilizando servicios que permitan mayor flexibilidad y disponibilidad.

Por ejemplo:

- Almacenamiento en AWS S3
- Procesamiento con AWS Glue o Lambda
- Orquestación con Step Functions

Esto permitiría construir un pipeline más robusto y preparado para producción.

Automatización del pipeline

El flujo actual se ejecuta de forma manual a través de un archivo principal. Como mejora, se podría incorporar una herramienta de orquestación como Apache Airflow.

Esto permitiría:

- Programar ejecuciones automáticas
- Gestionar dependencias entre tareas
- Monitorear el estado del pipeline

Despliegue del modelo (Model Serving)

El modelo actualmente se utiliza de forma local. Una mejora clave sería exponerlo como un servicio para su consumo en tiempo real.

Algunas opciones:

- Crear una API con FastAPI
- Contenerizar la aplicación con Docker
- Desplegar en servicios cloud

Esto permitiría integrar el modelo en aplicaciones reales.

Mejora del modelo

Desde el punto de vista de machine learning, se podrían implementar mejoras como:

- Ajuste de hiperparámetros (Grid Search / Random Search)
- Pruebas con otros modelos (XGBoost, LightGBM)
- Evaluación con técnicas más robustas

Esto permitiría optimizar el rendimiento del modelo.

8. Conclusión

El desarrollo de este proyecto permitió implementar un flujo completo de procesamiento de datos, desde la ingesta de información hasta la generación de predicciones mediante un modelo de machine learning. A lo largo del proceso, se aplicaron principios clave de ingeniería de datos, como la modularización del pipeline ETL, la separación de capas de datos y la incorporación de mecanismos de trazabilidad mediante logging.

Asimismo, la integración de técnicas de feature engineering y el uso de un modelo de clasificación permitieron abordar un problema real como la detección de fraude, demostrando cómo el análisis de datos puede generar valor en contextos financieros.

Este proyecto refleja la capacidad de diseñar soluciones estructuradas, escalables y orientadas a la resolución de problemas reales, combinando procesamiento de datos y modelado predictivo en un mismo flujo de trabajo.

En un entorno profesional, este tipo de solución podría integrarse como parte de sistemas de monitoreo de transacciones, contribuyendo a la identificación temprana de comportamientos anómalos y apoyando la toma de decisiones basada en datos.